

**РОССИЙСКИЙ УНИВЕРСИТЕТ ДРУЖБЫ
НАРОДОВ**

**Факультет физико-математических и естественных наук
Кафедра прикладной информатики и теории вероятностей**

**ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ № 8.**
Дисциплина: Архитектура компьютера

Студент: Павлушина Виктория Александровна

Группа: НКАбд-05-25

МОСКВА 2025 г.

Оглавление

1. Цель работы	3
2.Выполнение лабораторной работы	4
2.1. Реализация циклов в NASM	4
2.2. Обработка аргументов командной строки.....	7
3.Задание для самостоятельной работы	10
4.Вывод.....	12

1. Цель работы

Приобретение навыков написания программ с использованием циклов и обработкой аргументов командной строки.

2.Выполнение лабораторной работы

2.1. Реализация циклов в NASM

Создадим каталог для программ лабораторной работы № 8, перейдём в него и создадим файл *lab8-1.asm*:

```
vapavlushina@dk7n07 ~/work/arch-pc $ mkdir ~/work/arch-pc/lab08
vapavlushina@dk7n07 ~/work/arch-pc $ cd ~/work/arch-pc/lab08
vapavlushina@dk7n07 ~/work/arch-pc/lab08 $ touch lab8-1.asm
```

Рисунок 1 Создали каталог lab08, а в нём файл lab8-1.asm

Введём в файл *lab8-1.asm* текст программы из листинга 8.1. Создадим исполняемый файл и проверим его работу.

```
%include 'in_out.asm'

SECTION .data
    msg1 db 'Введите N: ',0h

SECTION .bss
    N:    resb 10

SECTION .text
    global _start
_start:

; ----- Вывод сообщения 'Введите N: '
    mov eax,msg1
    call sprint

; ----- Ввод 'N'
    mov ecx, N
    mov edx, 10
    call sread

; ----- Преобразование 'N' из символа в число
    mov eax,N
    call atoi
    mov [N],eax

; ----- Организация цикла
    mov ecx,[N]      ; Счетчик цикла, 'ecx=N'
label:
    mov [N],ecx
    mov eax,[N]
    call iprintLF    ; Вывод значения 'N'
    loop label       ; 'ecx=ecx-1' и если 'ecx' не '0'
                    ; переход на 'label'

    call quit
```

Рисунок 2 Текст листинга 8.1

```

vapavlushina@dk7n07 ~/work/arch-pc/lab08 $ nasm -f elf lab8-1.asm
vapavlushina@dk7n07 ~/work/arch-pc/lab08 $ ld -m elf_i386 -o lab8-1 lab8-1.o
vapavlushina@dk7n07 ~/work/arch-pc/lab08 $ ./lab8-1
Введите N: 6
6
5
4
3
2
1

```

Рисунок 3 Запустили исполняемый файл

Следующий пример показывает, что использование регистра *ecx* в теле цикла *loop* может привести к некорректной работе программы. Изменим текст программы, добавив изменение значение регистра *ecx* в цикле:

```

label:
    sub ecx,1      ; `ecx=ecx-1`
    mov [N],ecx
    mov eax,[N]
    call iprintLF
    loop label

```

Рисунок 4 изменения, которые мы должны выполнить в новом файле

Создадим файл *lab8-01.asm*, сделаем исполняемый файл и проверим его работу.

```

vapavlushina@dk7n07 ~/work/arch-pc/lab08 $ touch lab8-01.asm
vapavlushina@dk7n07 ~/work/arch-pc/lab08 $ nasm -f elf lab8-01.asm
vapavlushina@dk7n07 ~/work/arch-pc/lab08 $ ld -m elf_i386 -o lab8-01 lab8-01.o

```

Рисунок 5 Создали файл *lab8-01.asm*, оттранслировали

```

vapavlushina@dk7n07 ~/work/arch-pc/lab08 $ ./lab8-01
Введите N: 6
5
3
1

```

Рисунок 6 Запустили файл с чётным N

```
4294952416
4294952414
4294952412
4294952410
4294952408
4294952406
4294952404
4294952402
4294952400
4294952398
4294952396
1701057301
```

Рисунок 7 Вывод программы при нечетном N

Заметим, что количество выведенных значений не соответствует введенному N. Если ввести нечётное значение N, то будет некорректный долгий вывод, если же введём чётное число, то будут выводиться все нечётные значения от 1 до N.

Для использования регистра `ecx` в цикле и сохранения корректности работы программы можно использовать стек. Внесём изменения в текст программы добавив команды `push` и `pop` (добавления в стек и извлечения из стека) для сохранения значения счетчика цикла **loop**. Для этого создадим копию файла с именем *lab8-02.asm* :

```
label:
    push ecx                ; добавление значения ecx в стек
    sub ecx,1
    mov [N],ecx
    mov eax,[N]
    call iprintLF
    pop ecx                 ; извлечение значения ecx из стека

    loop label
```

Рисунок 8 Изменения программы

```
vapavlushina@dk7n07 ~/work/arch-pc/lab08 $ touch lab8-02.asm
vapavlushina@dk7n07 ~/work/arch-pc/lab08 $ nasm -f elf lab8-02.asm
vapavlushina@dk7n07 ~/work/arch-pc/lab08 $ ld -m elf_i386 -o lab8-02 lab8-02.o
vapavlushina@dk7n07 ~/work/arch-pc/lab08 $ ./lab8-02
Введите N: 6
5
4
3
2
1
0
```

Рисунок 9 Создание, транслирование и ввод четного значения в файл *lab8-02.asm*

```
vapavlushina@dk7n07 ~/work/arch-pc/lab08 $ ./lab8-02
Введите N: 3
2
1
0
```

Рисунок 10 Ввод нечётного значения

Программа выводит всё корректно и при вводе чётного, и при вводе нечётного значений.

2.2. Обработка аргументов командной строки

Создадим файл *lab8-2.asm* в каталоге *~/work/arch-pc/lab08* и введём в него текст программы из листинга 8.2.

Создадим исполняемый файл и запустим его, указав аргументы:

```
vapavlushina@dk7n07 ~/work/arch-pc/lab08 $ touch lab8-2.asm
vapavlushina@dk7n07 ~/work/arch-pc/lab08 $ nasm -f elf lab8-2.asm
vapavlushina@dk7n07 ~/work/arch-pc/lab08 $ ld -m elf_i386 -o lab8-2 lab8-2.o
vapavlushina@dk7n07 ~/work/arch-pc/lab08 $ ./lab8-2 1 2 3
1
2
3
```

Рисунок 11 Запустили файл *lab8-2.asm*

Все 3 аргумента были обработаны программой.

Рассмотрим еще один пример программы которая выводит сумму чисел, которые передаются в программу как аргументы. Создадим файл *lab8-3.asm* в каталоге *~/work/archpc/lab08* и введём в него текст программы из листинга 8.3.

```
vapavlushina@dk7n07 ~/work/arch-pc/lab08 $ touch lab8-3.asm
vapavlushina@dk7n07 ~/work/arch-pc/lab08 $ nasm -f elf lab8-3.asm
vapavlushina@dk7n07 ~/work/arch-pc/lab08 $ ld -m elf_i386 -o lab8-3 lab8-3.o
vapavlushina@dk7n07 ~/work/arch-pc/lab08 $ ./lab8-3 12 13 7 10 5
Результат: 47
```

Рисунок 12 Создание, транслирование и запуск файла *lab8-3.asm* с 5-ю аргументами

```
%include 'in_out.asm'
SECTION .data
msg db "Результат: ",0
SECTION .text
global _start
_start:
    pop ecx ; Извлекаем из стека в `ecx` количество
    ; аргументов (первое значение в стеке)
    pop edx ; Извлекаем из стека в `edx` имя программы
    ; (второе значение в стеке)
    sub ecx,1 ; Уменьшаем `ecx` на 1 (количество
    ; аргументов без названия программы)
    mov esi, 0 ; Используем `esi` для хранения
    ; промежуточных сумм
next:
    cmp ecx,0h ; проверяем, есть ли еще аргументы
    jz _end ; если аргументов нет выходим из цикла
    ; (переход на метку `_end`)
    pop eax ; иначе извлекаем следующий аргумент из стека
    call atoi ; преобразуем символ в число
    add esi,eax ; добавляем к промежуточной сумме
    ; след. аргумент `esi=esi+eax`
    loop next ; переход к обработке следующего аргумента
_end:
    mov eax, msg ; вывод сообщения "Результат: "
    call sprint
    mov eax, esi ; записываем сумму в регистр `eax`
    call iprintLF ; печать результата
    call quit ; завершение программы
```

Рисунок 13 Текст листинга 8.3

Изменим текст программы из листинга 8.3 для вычисления произведения аргументов командной строки. Для этого создадим копию файла *lab8-3.asm* с именем *lab8-31.asm*:


```
vapavlushina@dk7n07 ~/work/arch-pc/lab08 $ touch lab8-31.asm
vapavlushina@dk7n07 ~/work/arch-pc/lab08 $ nasm -f elf lab8-31.asm
vapavlushina@dk7n07 ~/work/arch-pc/lab08 $ ld -m elf_i386 -o lab8-31 lab8-31.o
vapavlushina@dk7n07 ~/work/arch-pc/lab08 $ ./lab8-31 12 13 7 10 5
Результат: 54600
```

Рисунок 14 Работа программы

```
%include 'in_out.asm'
SECTION .data
msg db "Результат: ",0
SECTION .text
global _start
_start:
pop ecx ; Извлекаем из стека в `ecx` количество аргументов
pop edx ; Извлекаем из стека в `edx` имя программы
sub ecx,1 ; Уменьшаем `ecx` на 1 (количество аргументов без названия программы)

mov esi, 1 ; Используем `esi` для хранения произведения (начинаем с 1, а не с 0)

next:
cmp ecx,0h ; проверяем, есть ли еще аргументы
jz _end ; если аргументов нет выходим из цикла
pop eax ; иначе извлекаем следующий аргумент из стека
call atoi ; преобразуем символ в число

; Умножаем текущее произведение на новый аргумент
imul esi, eax ; esi = esi * eax

loop next ; переход к обработке следующего аргумента

_end:
mov eax, msg ; вывод сообщения "Результат: "
call sprint
mov eax, esi ; записываем произведение в регистр `eax`
call iprintLF ; печать результата
call quit ; завершение программы
```

Рисунок 15 Листинг файла lab8-31.asm

3.Задание для самостоятельной работы

Напишем программу, которая находит сумму значений функции $f(x)$ для $x = x_1, x_2, \dots, x_n$, т.е. программа должна выводить значение $f(x_1) + f(x_2) + \dots + f(x_n)$. Значения x_i передаются как аргументы. Вид функции $f(x)$ выберем из таблицы 8.1 вариантов заданий в соответствии с вариантом, полученным при выполнении лабораторной работы № 6 (у нас это вариант 16). Создадим исполняемый файл и проверим его работу на нескольких наборах $x = x_1, x_2, \dots, x_n$.

$$16 \qquad 30x - 11$$

Рисунок 16 Функция для 16 варианта

```
vapavlushina@dk7n07 ~/work/arch-pc/lab08 $ touch sam.asm
vapavlushina@dk7n07 ~/work/arch-pc/lab08 $ nasm -f elf sam.asm
vapavlushina@dk7n07 ~/work/arch-pc/lab08 $ ld -m elf_i386 -o sam sam.o
vapavlushina@dk7n07 ~/work/arch-pc/lab08 $ ./sam 1 2 3 4
Результат: 256
```

Рисунок 17 Результат выполнения работы программы для самостоятельной работы

```
vapavlushina@dk7n07 ~/work/arch-pc/lab08 $ ./sam 12 13 14 15
Результат: 1576
vapavlushina@dk7n07 ~/work/arch-pc/lab08 $ █
```

Рисунок 18 Результат выполнения работы программы с другими аргументами

```

%include 'in_out.asm'
SECTION .data
msg db "Результат: ",0
error_msg db "Ошибка: недостаточно аргументов",0
SECTION .text
global _start

; Функция f(x) = 30x - 11
; Вход: eax = x
; Выход: eax = 30*x - 11
f:
    push ebx
    mov ebx, 30      ; ebx = 30
    imul ebx         ; eax = eax * 30 (теперь в eax результат)
    sub eax, 11      ; eax = eax - 11
    pop ebx
    ret

_start:
    pop ecx          ; Извлекаем количество аргументов
    pop edx          ; Извлекаем имя программы

    sub ecx, 1       ; Уменьшаем количество аргументов (без имени программы)
    jnz continue     ; Если есть аргументы, продолжаем

    ; Если аргументов нет
    mov eax, error_msg
    call sprintf
    call quit

continue:
    mov esi, 0       ; Используем esi для хранения суммы (esi = 0)

next:
    pop eax          ; Извлекаем следующий аргумент из стека
    call atoi        ; Преобразуем символ в число (результат в eax)

    call f           ; Вычисляем f(x) = 30*x - 11

    add esi, eax      ; Добавляем результат к сумме (esi = esi + f(x))

    loop next        ; Переход к обработке следующего аргумента

_end:
    mov eax, msg      ; Вывод сообщения "Результат: "
    call sprintf
    mov eax, esi      ; Записываем сумму в регистр eax
    call iprintLF     ; Печать результата
    call quit         ; Завершение программы

```

Рисунок 19 Текст программы sam.asm

4.Вывод

Приобрела навыки написания программ с использованием циклов и обработкой аргументов командной строки.